Section 3: Data Types & Constraints

3.1 Data Types

Data Type	Use Case / Description
INT / BIGINT	Whole numbers — age, count, ID
DECIMAL(p,s) / NUMERIC	Precise decimal — salary, price (p=total digits, s=after decimal)
FLOAT / REAL	Approximate decimal — scientific values
CHAR(n)	Fixed-length text — gender ('M'/'F'), country code
VARCHAR(n)	Variable-length text — names, emails (up to n chars)
NVARCHAR(n)	Unicode variable text — supports Hindi, Chinese etc.
TEXT	Large text blocks (avoid; use VARCHAR(MAX) instead)
DATE	Date only — '2024-06-15'
TIME	Time only — '14:30:00'
DATETIME / DATETIME2	Date + time — transactions, logs
BIT	Boolean — 1 (true) or 0 (false)
UNIQUEIDENTIFIER	GUID — globally unique ID

3.2 Constraints

Constraints Reference
PRIMARY KEY — Uniquely identifies each row; NOT NULL + UNIQUE; one per table
UNIQUE — All values in column must be distinct; NULLs allowed
NOT NULL — Column must always have a value
DEFAULT — Provides a default value when no value is given
CHECK — Validates data before inserting: CHECK (Age >= 18)
FOREIGN KEY — Links child table to parent; enforces referential integrity
IDENTITY(s,i) — Auto-increment column starting at s, incrementing by i

```
-- Constraints in practice
CREATE TABLE Employees (
  EmpID      INT           IDENTITY(1001, 1) PRIMARY KEY,
  Email      VARCHAR(100)  UNIQUE NOT NULL,
  Salary     DECIMAL(10,2) CHECK (Salary > 0),
  DeptID     INT           FOREIGN KEY REFERENCES Departments(DeptID),
  JoinDate   DATE          DEFAULT GETDATE(),
  IsActive   BIT           DEFAULT 1
);
```

Section 4: Filtering, Sorting & Clauses

4.1 WHERE Clause

Filters rows based on conditions. Works with all DML statements.

```
-- Equality and comparison
SELECT * FROM Employees WHERE Department = 'Sales';
SELECT * FROM Employees WHERE Salary > 80000;
SELECT * FROM Employees WHERE HireDate > '2020-12-31';
SELECT * FROM Employees WHERE Department != 'HR';   -- or <>
```

4.2 Logical Operators

```
-- AND, OR, NOT
SELECT * FROM Employees WHERE Department = 'IT' AND Salary > 60000;
SELECT * FROM Employees WHERE City = 'Delhi' OR City = 'Mumbai';
SELECT * FROM Employees WHERE NOT Department = 'Sales';
```

4.3 Special Operators

```
-- IN — match any value in a list
SELECT * FROM Employees WHERE Department IN ('IT', 'Sales', 'HR');

-- BETWEEN — range (inclusive)
SELECT * FROM Employees WHERE Salary BETWEEN 50000 AND 100000;

-- LIKE — pattern matching
SELECT * FROM Employees WHERE Name LIKE 'A%';     -- starts with A
SELECT * FROM Employees WHERE Email LIKE '%@gmail.com';
SELECT * FROM Employees WHERE Name LIKE ' a%';    -- 2nd char is 'a'

-- IS NULL / IS NOT NULL
SELECT * FROM Employees WHERE ManagerID IS NULL;
SELECT * FROM Employees WHERE PhoneNumber IS NOT NULL;
```

4.4 DISTINCT

```
-- Get unique values
SELECT DISTINCT Department FROM Employees;
SELECT DISTINCT City, Department FROM Employees;
```

4.5 ORDER BY

```
-- Sort results
SELECT * FROM Employees ORDER BY Salary DESC;
SELECT * FROM Employees ORDER BY Department ASC, Salary DESC;

-- TOP — limit results (MSSQL)
SELECT TOP 10 * FROM Employees ORDER BY Salary DESC;
SELECT TOP 5 PERCENT * FROM Employees ORDER BY HireDate DESC;
```

4.6 CASE Expression

The CASE expression adds conditional logic inside a SELECT query — like an if-else in SQL.

```
-- Simple CASE
SELECT Name, Salary,
  CASE
    WHEN Salary > 100000 THEN 'High'
    WHEN Salary BETWEEN 50000 AND 100000 THEN 'Medium'
    ELSE 'Low'
  END AS SalaryBand
FROM Employees;

-- CASE in UPDATE
UPDATE Employees
SET Grade = CASE
  WHEN Salary > 100000 THEN 'A'
  WHEN Salary > 60000  THEN 'B'
  ELSE 'C'
END;
```

4.7 NULL Handling Functions

```
-- ISNULL — replace NULL with a value
SELECT Name, ISNULL(PhoneNumber, 'N/A') AS Phone FROM Employees;

-- COALESCE — returns first non-NULL value
SELECT COALESCE(Mobile, HomePhone, 'No Contact') AS Contact FROM Employees;

-- NULLIF — returns NULL if two values are equal
SELECT NULLIF(Score, 0) FROM Results;  -- returns NULL instead of 0
```